

The Easy Knot: Computationally Implementing Reidemeister Moves for Unknotting

Senior Honor Thesis

Brandeis University

Spring 2025

Ephraim Zimmerman

Acknowledgments

Thank you to my professors who guided me initially: Daniel Ruberman, for introducing me to knot theory; Antonella Di Lillo, for helping me get started with this paper.

Thank you to Tim Hickey, for your expert guidance and helping to bring everything together.

Abstract

Picture a very large bundle of knotted string. Could you untie it into a straight lace? How many twists and loops might it take to do so? Although simple in concept, the problem of determining whether a knot can be untangled within a bounded number of moves, and whether it represents another knot, has been proven to be NP-hard [1]. Existing tools for exploring knots primarily focus on identifying knots through invariants and visualizations, making transformations largely manual and difficult to execute. This paper explores the components necessary to reframe the problem through a novel data structure called the Easy Knot. This structure is designed to parametrize knots in a way that enables automated generation, untangling, and comparison, thereby providing a new perspective on the computational complexity of knot simplification and equivalence. 

1 Creating the Easy Knot

A brief history

The concepts behind knot theory span over hundreds of years, but it became first formally established when Carl Friedrich Gauss', in the 18th century, began studying the linking of closed curves. In the nineteenth century, physicist William Thomson proposed a theory that states that some atoms can become knotted, which he referred to as the “vortex atom theory” [2]. Thomson’s hypothesis motivated Peter Guthrie Tait to compile extensive tables of knots, classifying them based on the crossings of their strands. This became known as the Tait conjectures [3], which are a series of solved components that all knots contain. Tait’s work marked the beginning of knot tabulation, introducing concepts such as a projection of a knot and being able to manipulate the crossing of its strands without changing the classification of the knot itself. The 20th century would mark a more formal beginning to the foundation for knot theory as a field. J. Alexander introduced the Alexander polynomial invariant for knots, which provided mathematicians with the first algebraic tool to distinguish knots [4]. The advancement of combinatorial-based mathematics in topology were also used for understanding certain properties of knots and how they can be deformed.

1.1 What is a knot?

The term “knot” in pure-mathematics has a specific definition that differs slightly from what is commonly used in practice. The practical use of a knot is used to fasten something together in a way that prioritizes its ability to be intentionally untied into a straight lace. However, in order for a rope to be considered a mathematical knot, for example, you would need to glue the two ends of it together. This would create a closed curve that cannot be untangled. This structure is referred to as the unknot, the most trivial that a knot can be.

The basic principals of knot theory tell us that there exist knots that are not the unknot. A simple way to prove this theorem is by coloring the strands of a knot and changing color when the strand you are following goes under another strand. If there are at least two colors and no more than three, the knot is tri-colorable. In the following two drawings, the leftmost knot is the unknot twisted upon itself. It does not meet the first rule of having at least two colors. If we draw the two colors, the colored strands will collide. The rightmost knot, the trefoil, follows that rule. Therefore, there exists a knot that is not trivial.



Figure 1: Tricolorability test on unknot (left) the trefoil (middle), chiral version of middle knot (right)

This is what’s known as an invariant, and is used to distinguish between two knots: if a knot is tricolorable, it cannot not be the unknot. In developing a recursive algorithm that is tasked with distinguishing and deforming knot diagrams, this is a valuable base.

In many code-based representations of knot diagrams, knots typically get modeled using graph data structures, where edges represent strand segments and graph nodes are the crossings. This is useful for fast traversal of the knot, easy encoding of crossing relationships, and is simple to visualize in 2-dimensions. However, one major limitation of this approach is the loss of over-under information at each crossing. For instance, in a three-dimensional representation of a knot, simply reversing the knot's orientation would leave a single node unable to convey which strand passes over and which passes under.

This concept is easily illustrated with a flat diagram. As shown in Figure 1, knots are typically represented as projections unto a two-dimensional surface, where crossings are drawn with the upper strand appearing continuous. In higher-dimensional space, however, the way a knot is presented becomes more significant, as there are infinitely many possible projections, some of which can disrupt the original coloring scheme.

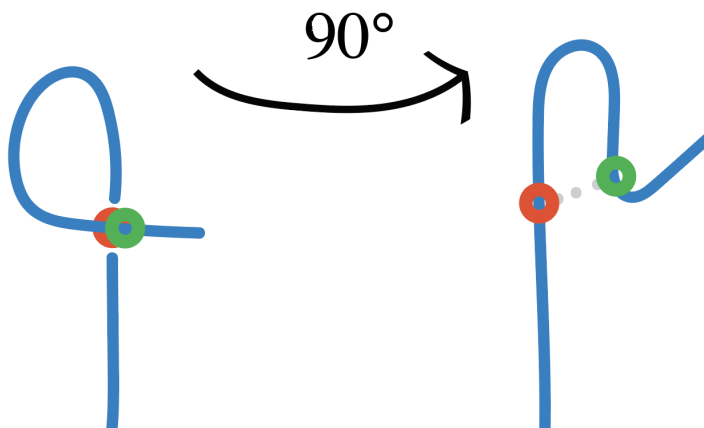


Figure 2: Positive and negative crossing points on the same loop

This leads to a major component of the Easy Knot: sibling nodes. For each crossing, two distinct points of interest are defined: one representing the over-crossing strand and the other representing the under-crossing strand. These are labeled with a positive (green or “+”) or negative (red or “-”) sign. While these labels are technically arbitrary, they provide consistency and indicate the forward direction within the data structure. Since siblings are connected inherently in the data structure, a visual proximity problem does not exist.

Another loss when working with graphs is seen when describing amphichirality, the process of using a series of deformations that swap crossings in such a way that you achieve the mirrored image of the knot. [6] The property of a knot being indistinguishable from its mirror image plays a crucial role in understanding knot complexity. Formally, for a knot K to be amphichiral, its mirror image K^* must be able to be continuously deformed into K (it is ambient isotopic). This property has deep implications for classifying knots, as it directly impacts the invariants used to distinguish knots between each other.

The last component to note is the direction of the knot and where the data structure begins. In general, the paper provides examples from the highest positive node, and it will travel clockwise from it. As long as the direction never changes while modifying the knot, the starting direction is arbitrary.

1.2 Transformations

Since the fundamentals of what makes a knot have been defined, we can begin to introduce a code-based representation of these concepts with the end goal of showcasing how difficult it is to unknot and compare knots with each other. The Easy Knot stores four key pieces of information for each crossing: its positive or negative sign, the preceding crossing, the following crossing, and its sibling crossing. Together, this information creates a web of directed crossings. There are many ways of implementing this logic depending on your preferred language. Python has been chosen for its simplicity, in this example it utilizes dictionaries to index knot data.

```
class EastKnot: 
    """
    Each entry in the structure dictionary represents a node with:
    - sign ("+" or "-")
    - backward connection
    - forward connection
    - sibling node knot ID
    """

    def __init__(self, structure):
        """
        Initialize the Knot instance.

        Parameters:
        structure (dict): Dictionary representing the knot structure.
        """
        self.structure = structure

    def is_unknot(self):
        """
        Determine whether the knot structure represents an unknot.

        Returns:
        bool: True if the structure is considered an unknot, False otherwise.
        """
        # Extract the set of all crossing signs
        signs = {node[0] for node in self.structure.values()}

        # Extract the set of numeric values (excluding the sign)
        numeric_values = {tuple(node[1:]) for node in self.structure.values()}

        # Trivial case: fewer than 6 nodes is always an unknot
        if len(self.structure) < 6:
            return True

        # Unknot condition: all signs are the same or all numeric
        # connections are the same
        return len(signs) == 1 or len(numeric_values) == 1

# Example knot structures

# Completely empty structure (trivial unknot)
unknot = Knot({})
```

The above code contains a function to determine if a given knot is the unknot. The purpose of this is primarily to check if the knot is simple enough to reduce to an empty dictionary. There are many ways that this can be accomplished, but for simplicity, the requirement is that at least 6 total nodes exist. An example of an unknot that would be recognized as an empty dictionary would be the following:

- If we generate an unknot twisted a single time (reference back to Figure 1), the starting dictionary would appear as such:

$$0 : [" + ", 1, 1, 1], 1 : [" - ", 0, 0, 0]$$

This is the unknot, since we know the trefoil is the least trivial non-trivial knot, and it contains at least three crossings. This example is the most trivial that the unknot can be other than an empty dictionary. Reducing to the unknot automatically is helpful if the optimization goal is to find an empty dictionary.

1.2.1 Reidemeister I

An crucial concept in working with the physical constraints of knots are the techniques used to untangle them. Kurt Reidemeister stated the three basic principles as: twisting, poking, and sliding, now all referred to as Reidemeister moves I, II and III, respectively [5].

When considering the basics of a useful data structure for a knot, it became helpful to start by designing the parameters in terms of how the Reidemeister moves would be most simply implemented. Once Reidemeister moves are baked into the data structure, it can be assumed many other knot properties are inherited alongside them.

The first Reidemeister move is described as twisting, untwisting, and straightening a strand over itself. In the following figure, Reidemeister I is demonstrated on an isolated and singular strand, where the crossings swap three times:

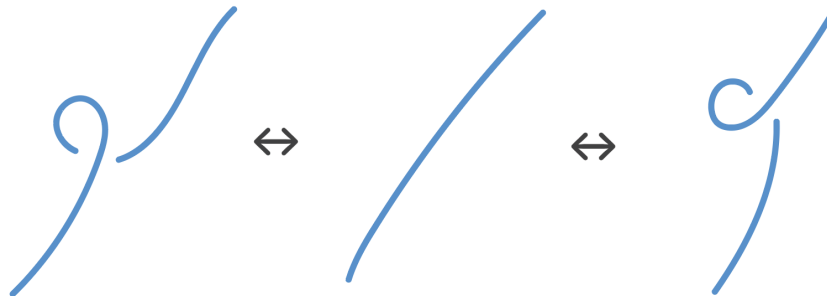


Figure 3: Reidemeister I

The leftmost and rightmost drawings represent a reversal of the strand looping over itself in opposite directions. These two representations must be defined in order for the knot to exist in 3-dimensional space, as they are all equivalent. The middle image represents a simplification of the loop, where the crossing is completely removed. All three of these images represent the same construct in different forms.

To identify if a Reidemeister I move is possible, the logic is if a given crossing forms a loop with its own sibling node, it is able to be unlooped. Specifically, it evaluates whether the sibling node of the crossing is

also the node to which it connects forward. This condition indicates that the crossing and its sibling create an isolated loop.

While moving certain strands might be physically possible, Reidemeister moves operate in a generalized enough way for them to work even when the rest of the knot is unknown. If, for example, a strand passes under the loop, the move can still be executed since the relationship both siblings have to the other strand are both negative. Reidemeister I could not be performed if a strand passes through the loop, however, because both sibling nodes are neighboring the strand and have a positive and negative relationship to it. The following figure demonstrates these two examples:

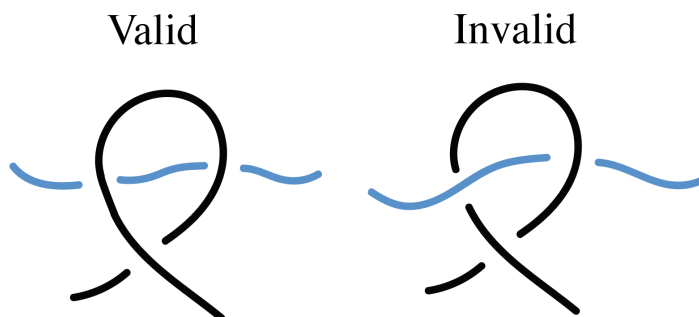


Figure 4: Reidemeister I move validity examples

In the example above, the blue horizontal strand represents an arbitrary crossing. One might wonder whether it is possible to ignore Reidemeister I (for the invalid loop) by opening the loop, from left to right, and freeing the poked blue strand. To address this question, we can reconnect the strands to form a continuous path. The product turns out to form the trefoil knot and shows we can't actually free that strand:



Figure 5: Reconnected invalid Reidemesiter I use case

Reidemeister moves are generalized enough to not impact the underlying structure of the knot while still being able to perform deformations when lacking knowledge of the underlying structure. The Easy Knot implementation follows this exactly. Given a crossing, it checks whether the loop can be removed. If so, it will pop the crossing as well as it's underlying sibling, and reconnect the nodes to their neighbors.

Although a strand passing beneath a loop is easy to comprehend and visualize on paper, the situation becomes more complex when approached algorithmically. This challenge provides a clear and straightforward example of the difficulties involved in creating an unknotting algorithm. The leftmost image in Figure 4 shows a configuration that appears relatively simple to untie. From an algorithmic perspective, the data structure would interpret a strand located between sibling nodes as requiring the same sign between those

nodes. This sign indicates whether the strand lies above or below. The challenge becomes more difficult when multiple strands cross over the same area, raising the question of how such a situation can be implemented effectively. Figure 6 demonstrates this visually using multi-colored strands:



Figure 6: A very hard Reidemeister I move



None of the crossing strands are poked inside of the loop, they sit either on top or below it. Algorithmically, representing and tracking the spatial relationships between multiple strands, especially when they cross or are nested, requires maintaining an extremely combinatorially complex data structure. When multiple strands cross a region, as in Figure 6, the number of possible configurations that must be considered grows rapidly. The problem becomes not just identifying a correct sequence of moves, but efficiently navigating a vast solution space, where each choice may lead closer to or further from a solution. Demonstrating this with the Catalan number is particularly useful in this example. The Catalan number counts the ways in which some recursive structured are formed. The n th Catalan number is the number of correctly parenthesized expressions which can be mapped onto knots:

$$\frac{2(n)!}{(n+1)! * n!}$$

Which shows how the complexity grows quickly for each number of crossings n [6].

Basic situations can still be implemented algorithmically. The following Python code implements the reduction of a loop using a Reidemeister I move:

```
def reidemeister_1_simplify(self, crossing):

    # Easy Knot key:
    # index 0: node sign
    # index 1: crossing from back direction
    # index 2: next crossing
    # index 3: sibling node

    if crossing not in self.structure:
        return self.structure # Crossing not found; no operation performed.

    crossing_node = self.structure[crossing]
    sibling_crossing_number = crossing_node[3]
    sibling_node = self.structure[sibling_crossing_number]

    # Check connection direction to determine the type of twist.
    if crossing_node[3] == crossing_node[1]:
        # Right twist case: sibling connected backward.
        self.structure[sibling_node[1]][2] = crossing_node[2]
        self.structure[crossing_node[2]][1] = sibling_node[1]

        self.structure.pop(crossing)
        self.structure.pop(sibling_crossing_number)

        print(f"Performed right-twist Reidemeister I move at crossing {crossing}.")

    elif crossing_node[3] == crossing_node[2]:
        # Left twist case: sibling connected forward.
        self.structure[crossing_node[1]][2] = sibling_node[2]
        self.structure[sibling_node[2]][1] = crossing_node[1]

        self.structure.pop(crossing)
        self.structure.pop(sibling_crossing_number)

        print(f"Performed left-twist Reidemeister I move at crossing {crossing}.")

    # After simplifying, check if the structure is now the unknot.
    if self.is_unknot():
        self.structure = {}
    return self.structure
```

The code checks to ensure that, given a crossing, the next node is its sibling. The data structure is built upon a clockwise rotation base (when selecting a positive crossing, trace to the right). If this is valid, it removes the crossing from the data structure, connecting the sibling's subsequent node with the given crossing previous node.

1.2.2 Reidemeister II

Reidemeister II is described as moving a loop completely over another loop. Unlike Reidemeister I, the strand is not looped over itself. There is a bit of technical complexity in implementing a Reidemeister II move because it requires the existing strands to be connected to each other based on the directional flow of the knot, and keeping track of that requires some attention.

Given two crossings, a Reidemeister II move can be performed if

1. Either crossing #1's next crossing is crossing #2 and crossing #2's previous is crossing #1, or vice versa.
2. Both crossings have the same crossing sign (+ or -)

Once the move is completed, given that there are no extraneous strands as demonstrated with the Catalan example, the deleted nodes will transfer their and their sibling's data to the other crossings. With the following braid-like unknot structure, a random point is chosen to represent crossing #0. While crossings are indexed as integers increasing by 1, their numbers are not actually representative of anything. Any digit could be used as long as it does not change. In this case, the node number and their siblings are tabulated from top to bottom even though they are not necessarily in that sequential order.

```

braid = {
  0: ["+", 1, 3, 1], # arbitrarily start at the first positive loop crossing
  1: ["-", 2, 0, 0],
  2: ["+", 4, 1, 3], # index 0: node sign
  3: ["-", 0, 5, 2], # index 1: crossing from back direction
  4: ["+", 7, 2, 5], # index 2: next crossing
  5: ["-", 3, 6, 4], # index 3: sibling node
  6: ["+", 5, 7, 7],
  7: ["-", 6, 4, 6],
}

```

In the following example, the number sitting higher than it's pair is the positive crossing in the sibling pair. A Reidemeister II move can be applied to crossings #2 and #4, then #0 and #6.

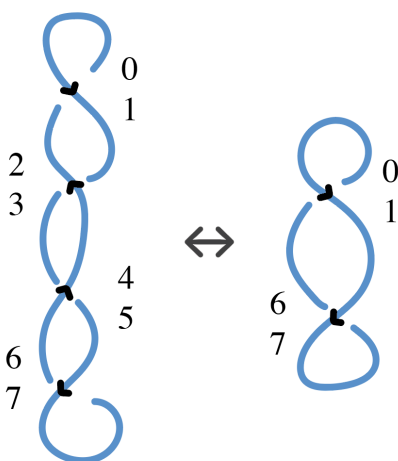


Figure 7: Reidemeister II move on braided unknot

```

def reidemeister_2_simplify(self, crossing1, crossing2):

    crossing1_next = self.structure[crossing1][2]
    crossing1_previous = self.structure[crossing1][1]
    crossing1_sibling = self.structure[crossing1][3]
    crossing2_sibling = self.structure[crossing2][3]
    crossing2_next = self.structure[crossing2][2]
    crossing1_sibling_previous = self.structure[crossing1_sibling][1]
    crossing2_sibling_next = self.structure[crossing2_sibling][2]

    if crossing1 in self.structure:
        if (crossing2 in self.structure and
            crossing1_next in self.structure and
            crossing2_next in self.structure and
            crossing1_previous in self.structure and
            crossing1_sibling in self.structure and
            crossing2_sibling in self.structure and
            crossing1_sibling_previous in self.structure and
            crossing2_sibling_next in self.structure):

            are_crossings_connected = (
                self.structure[crossing1][2] == crossing2 and
                self.structure[crossing2][1] == crossing1
            )

            have_same_sign = self.structure[crossing1][0] == self.structure[crossing2][0]

            # If both conditions are met, perform R2
            if are_crossings_connected and have_same_sign:
                """Modify crossings and their siblings so the next
                and previous nodes are connected to each other instead of them."""

                # Sliding out strand
                self.structure[crossing2_next][1] = crossing1_previous
                self.structure[crossing1_previous][2] = crossing2_next
                self.structure[crossing1_sibling_previous][2] = crossing2_sibling_next
                self.structure[crossing2_sibling_next][1] = crossing1_sibling_previous

                self.structure.pop(crossing1)
                self.structure.pop(crossing2)
                self.structure.pop(crossing1_sibling)
                self.structure.pop(crossing2_sibling)

                print(f"Performed Reidemeister II on {crossing1} and {crossing2}")
            return self.structure

```

Since the final result is a doubly-twisted unknot, another Reidemeister II move could be performed to reduce it to its final circular form. Alternatively, two Reidemeister I moves could be done on the crossings. Either way, since there are only two crossings, it automatically becomes an empty dictionary.

1.2.3 Reidemeister III

Reidemeister III is the last move and is defined as moving a strand completely over or under a crossing, commonly referred to as "poking" it through. This requires the attention of three total strands, with sibling nodes being removed and added to an entirely new strand. The following example shows a basic case of this in action:

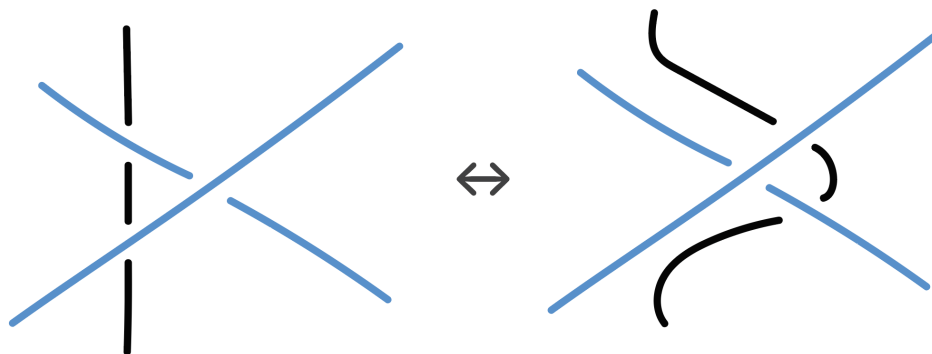


Figure 8: Reidemeister III poking example, black strand is pulled under other strands.

The black strand is being pulled through below the top two crossings. Reidemeister III is important to define because it does not add or remove any crossings and does not change the local sign of any crossing. While Reidemeister II is important for simplifying a knot by removing redundant crossings and reducing overall complexity, Reidemeister III is essential for proving knot equivalence when knots are the same but are arranged differently.

While this might seem trivial at first glance, the fact that a strand can be poked in such a way logically defines a crucial bit of information in pulling non-interlacing strands apart. There are many cases for which pulling a strand may limit a large number of other moves from being made.

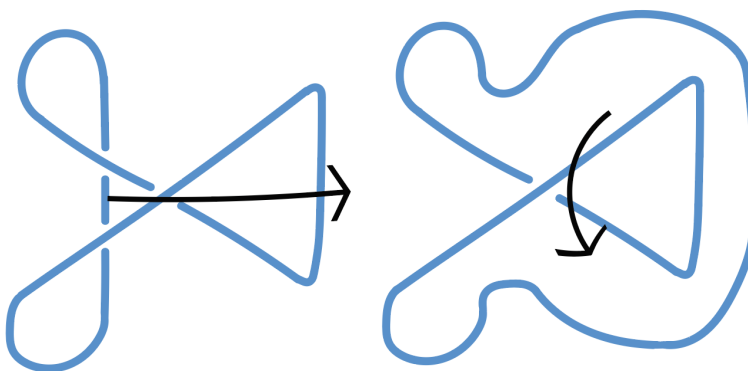


Figure 9: Reidemeister III and Reidemeister I moves applied successively

The above example utilizes Reidemeister III to pull a strand completely outside of two crossing strands. The following right-most image then performs a Reidemeister I move to complete the deformation.

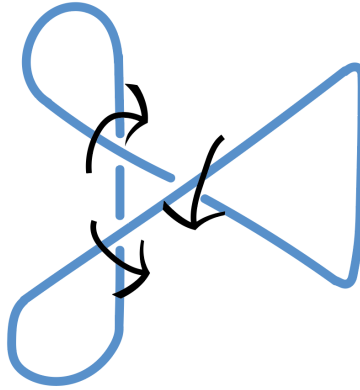


Figure 10: Three Reidemeister I moves applied to a knot

Alternatively, three separate Reidemeister I moves could be used to untangle the knot. On a large scale, Reidemeister III can significantly reduce the complexity of a knot by pulling a strand from beneath others.

From an implementation perspective, determining whether a strand can be pulled all the way through is a nontrivial task and would take an extremely large amount of computation. Intuitively, the key factor is that the two relevant crossings share the same sign. As long as this condition is met, the strand can continue to pass beneath other strands that also has the same sign, allowing it to be pulled through without contradiction. The following sample code is provided with a generic listing of passed crossings, it does not calculate those here.

```
def reidemeister_3_slide(self, crossing1, crossing2, passed_crossings: list):

    # Identify the sibling nodes associated with each crossing
    crossing1_sibling = self.structure[crossing1][3]
    crossing2_sibling = self.structure[crossing2][3]

    # Check if both crossings share the same sign
    can_pass = True
    if self.structure[crossing1][0] == self.structure[crossing2][0]:
        for crossing in passed_crossings:
            # All intermediate crossings must have the same sign
            if self.structure[crossing][0] != self.structure[crossing1][0]:
                can_pass = False
                break

    if can_pass:

        # not shown: reconnect nodes on strand where crossings
        # were popped

        # If all checks pass, remove the two crossings and their siblings
        self.structure.pop(crossing1_sibling)
        self.structure.pop(crossing2_sibling)
        self.structure.pop(crossing1)
        self.structure.pop(crossing2)
```

2 Brute-force unknotting



All three Reidemeister moves have been implemented in some form within this work. While the use-cases provided are intentionally generalized, the described techniques are sufficient to support a brute-force approach to untangling given knots. Moreover, due to the basic and consistent structural components of the Easy Knot, creating unknots from scratch becomes a relatively straightforward task. Clearly, Reidemeister moves operate most effectively when used in combination, as the application of one move can either enable or obstruct another. While the individual moves are conceptually straightforward, determining the correct sequence to simplify a knot may require some intuitive judgment or simply trial and error.

2.0.1 Knot generator

Since crossing nodes are connected forwards and backwards, it is possible to construct an automated knot generator given some crossing number. The ordering of the “next node chosen” can also be modified in a way to generate exclusively trivial or sometimes nontrivial knots.

The nodes are created as sibling pairs, each randomly assigned a negative or positive crossing value. In order for the knot to stay trivial, the way in which they are connected needs to be carefully constructed so the nodes are not looped through each other. For all knots, the last node generated must always reconnect to the beginning node of the knot. If this does not happen, and the start and end nodes are chosen at random, for larger knots it is possible that a strand will become trapped in between several crossings and cannot go back to the first node. The following code implements this in Python.

```
def generate_random_unknot(crossings):
    knot = {}
    crossing_ids = []
    signs = ["+", "-"]
    # Create nodes (with + and - crossings) in a paired fashion
    for crossing in range(crossings):
        # Each crossing and its opposite will form the unknot
        sign_1 = random.choice([0, 1])
        sign_2 = 1-sign_1
        knot[crossing] = [signs[sign_1], None, None, (2*crossings)-1-crossing] # + crossing
        knot[(2*crossings)-1-crossing] = [signs[sign_2], None, None, crossing] # - crossing

    for i in range(len(knot)):
        current = i % len(knot)
        next_node = (i + 1) % len(knot)
        knot[current][2] = next_node
        knot[next_node][1] = current

    # Close the loop (final crossing connects back to the first)
    first = 0
    last = len(knot)-1
    knot[last][2] = first
    knot[first][1] = last

    return knot
```

The following example illustrates what a knot with 3 crossings would look like. Keep in mind that this means 6 total nodes are generated. The most simple way to draw a knot from this generator is to write down on paper the positive and negative siblings in a fraction form. Then, draw from a node to its next node (2nd index) and account for if it's negative or positive:

$$0 : [", +", 5, 1, 5], 1 : [", +", 0, 2, 4], 2 : [", -", 1, 3, 3], 3 : [", +", 2, 4, 2], 4 : [", -", 3, 5, 1], 5 : [", -", 4, 0, 0]$$

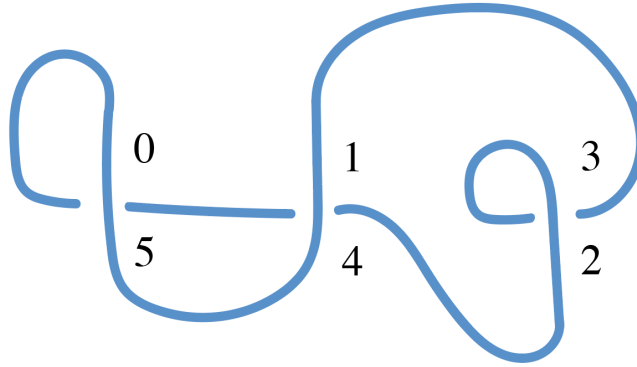


Figure 11: Generated knot with 3 crossings

This works well because it will inherently generate only Reidemeister I and II moves. In the simple case above, there are multiple ways to untie it: two Reidemeister I moves can be made for siblings 0/5 and 3/2, or a Reidemeister II move can be made along crossings 0 and 1. Another example would be for a single crossing, which always generates:

$$0 : [", +", 1, 1, 1], 1 : [", -", 0, 0, 0]$$

This is the only way to represent the unknot with a pair of nodes (other than a blank dictionary). In the data structure, the crossing nodes are connected to each other as such:

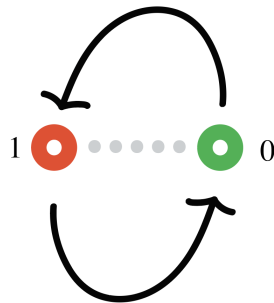


Figure 12: A single node pair forms the unknot

Now that a knot generator has been constructed, the given knots can be entered directly into an untangling algorithm, which is covered in the next section.

2.0.2 Untangling implementation

When implementing a brute-force algorithm, exactly like implementing individual Reidemeister moves, it is important to keep in mind the time complexity necessary to untangle these knots. Since the knots are only built with Reidemeister I and II moves they should be untied completely, however the complexity of each knot being untied will grow very quickly.

In the case of the Easy Knot, Reidemeister moves have been implemented under simplified constraints that do not account for all possible interactions between crossing strands. However, because the generator constructs knots with crossing nodes inherently connected, it is still possible to attempt untangling these very complex knots.

We will approach the problem by looking at each crossing individually to check if a Reidemeister I move can be made in a given crossing, then if a Reidemeister II move can be performed.

```
def untangle(self):
    change_made = True
    while change_made:
        change_made = False
        for crossing1 in list(self.structure.keys()):
            copy = self.structure
            if self.reidemeister_1_simplify(crossing1) != copy:
                change_made = True
        for crossing2 in list(self.structure.keys()):
            copy = self.structure
            if (crossing1 != crossing2 and
                self.reidemeister_2_simplify(crossing1, crossing2) != copy):
                change_made = True
```

This algorithm does not implement Reidemeister III since a proper implementation would have it checking every possible combination of neighboring crossings each time.  order to do this just with triplets it would increase number of operations to be incredibly high and very difficult to test results with.

In the current implementation, the worst-case scenario time complexity of the brute-force algorithm is $O(n^2)$. It appears  the first valid operation it encounters without considering alternative sequences of Reidemeister I or II.

2.0.3 Testing the untangling algorithm

Running this on very large unknots demonstrates that while the knots themselves are relatively simple, only containing loops and twists, it becomes highly computationally expensive since untwisting a certain loop may create other loop from it. We know this to be true since the brute force algorithm only stops when no moves are able to be made. The following graph shows the quadratic nature of this problem:

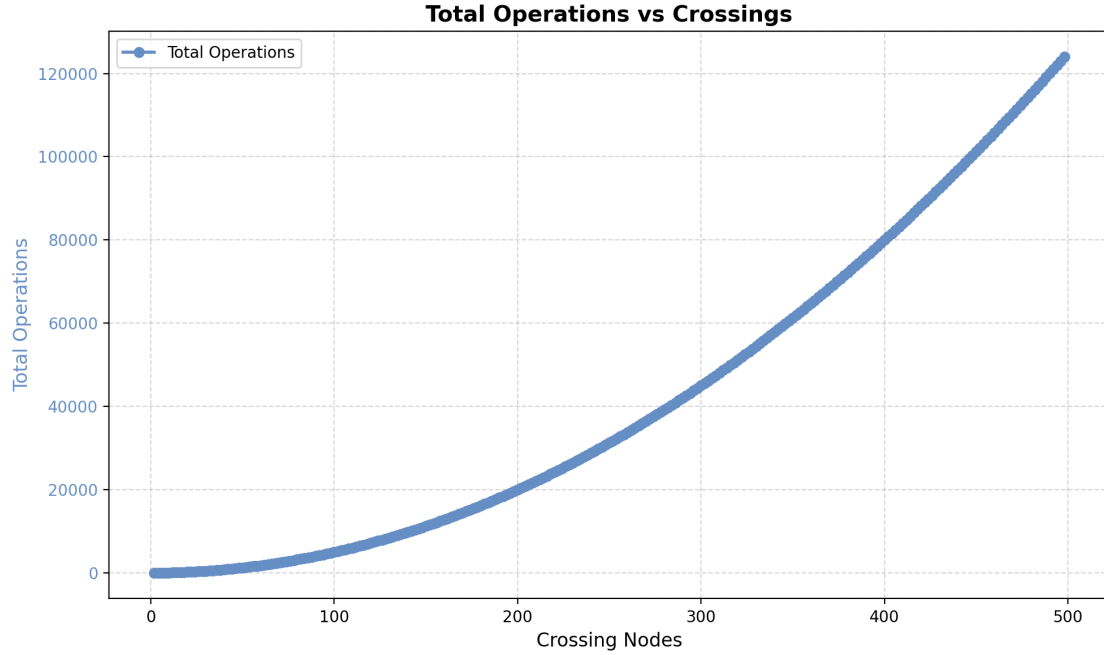


Figure 13: A single node pair forms the unknot

Statistic	Value
Min operations	2
Max operations	123998
Mean operations	41579.35
Standard deviation	37109.07
R^2 value	0.9380

Table 1: Statistical summary for operations needed

While the brute-force method handles knots with fewer than 10 crossings efficiently, even these simple loops become increasingly costly to resolve as the number of crossings grow. The quadratic fit tells us that larger knots require disproportionately more effort to untangle as untangling one loop may accidentally generate another.

3 Discussion

There are many applications that such a data structure can have both in theoretical and practical applications. In the field of autonomous robotics, for example, imagine a robotic system that must remove, clean, and re-lay entangled segments of wiring. An optimized algorithm that implements the Easy Knot could help calculate the optimized steps in advance to automate the untangling process while reducing manual input from a human. In knot theory, the Easy Knot could be implemented to aid in testing conjectures or classifying extremely complex non-trivial knots.

The goal of this paper is to describe the process of creating a useful knot data structure while highlighting the difficulty of the unknotting problem. In a sense, the unknotting problem helps to build on the foundation for which computational topology and algorithmic knot theory can be implemented computationally. By knowing the limitations of the problem, i.e., what is feasible, rules are provided in a way that helps build on the practical applications of it. Hopefully, this paper can serve as a guide for future enhancements of data structures in applied knot theory, as well as to serve as a demonstration to their implementation.

4 Future work

A future area of focus could be the development of alternative untangling algorithms which optimize the number of operations necessary to untangle the knot. Keeping a database of what has been untangled, for example, could be implemented to prevent the algorithm from continuously making the same mistake. More complex knot generators could also be derived, one that folds the knot over itself, for example, hence creating moves only untangled with Reidemeister III moves. Additional knot generators can also be implemented with more specificity in the types of crossings added. This logic can also be taught as an example of an abstract data structure in computer science education, demonstrating how to implement a 3-dimensional object and interact with its components.

References

- [1] Mesmay, Arnaud de, Yo'av Rieck, Eric Sedgwick, and Martin Tancer. “The Unbearable Hardness of Unknotting.” *Advances in Mathematics* 381 (April 2021): 107648. <https://doi.org/10.1016/j.aim.2021.107648>.
- [2] Silliman, Robert H. “William Thomson: Smoke Rings and Nineteenth-Century Atomism.” *Isis* 54, no. 4 (1963): 461–74. <http://www.jstor.org/stable/228151>.
- [3] May, K. O. (1967). THE ORIGIN OF THE FOUR-COLOR CONJECTURE. *The Mathematics Teacher*, 60(5), 516–519. <http://www.jstor.org/stable/27957611>
- [4] Kinoshita, S. (1961). On the Alexander Polynomials of 2-Spheres in A 4-Sphere. *Annals of Mathematics*, 74(3), 518–531. <https://doi.org/10.2307/1970296>
- [5] Trace, Bruce. “On the Reidemeister Moves of a Classical Knot.” *Proceedings of the American Mathematical Society* 89, no. 4 (1983): 722–24. <https://doi.org/10.2307/2044613>.
- [6] Campbell, Douglas M. “The Computation of Catalan Numbers.” *Mathematics Magazine* 57, no. 4 (1984): 195–208. <https://doi.org/10.2307/2689673>.

Adams, Colin C. *The Knot Book: An Elementary Introduction to the Mathematical Theory of knots* was also used heavily as a foundation for which this paper was based. The inspiration for this thesis came from page 4, where Adams lists a unsolved problem as: Write a computer algorithm that can tell whether a knot that is given is the unknot.